

Интерпретируемый C++: Развертывание интерактивной среды xeus-cling в Jupyter Notebook

C++ как Python: Превращаем компилируемый язык в
интерактивный инструмент

Зачем C++ быть интерактивным?

Идея заключается в том, чтобы перенести удобство Python (ячейки, мгновенный вывод) в мир высокопроизводительного C++

Технология: Cling — интерпретатор C++ на базе LLVM, разработанный в CERN для обработки данных Большого адронного коллайдера

Как это устроено (Архитектура)

Kernel (Ядро): В Jupyter мы используем ядро xeus-cling. Оно выступает «переводчиком» между интерфейсом блокнота и интерпретатором Cling

JIT-компиляция: Код не компилируется в бинарный файл целиком. Каждая ячейка компилируется «на лету» (Just-In-Time) и добавляется в текущую сессию памяти

Глобальное состояние: Если объявлено `int x = 10;` в одной ячейке, переменная `x` будет доступна во всех последующих ячейках

Подготовка среды

```
# 1. Установка condacolab для быстрой настройки среды
!pip install -q condacolab
import condacolab
condacolab.install()
```

... Downloading https://github.com/jaimergp/miniforge/releases/download/24.11.2-1_colab/Miniforge3-colab-24.11.2-1_colab-Linux-x86_64.sh...
Installing...
Adjusting configuration...
Patching environment...
Done in 0:00:12
Restarting kernel...

Подготовка "фундамента": установка инструментов для развертывания сторонних ядер (Kernels) в облаке Google Colab

Подготовка среды

```
# Автоматическая установка
!conda install xeus-cling -c conda-forge -y

... Channels:
  - conda-forge
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 24.11.3
  latest version: 26.1.1

Please update conda by running

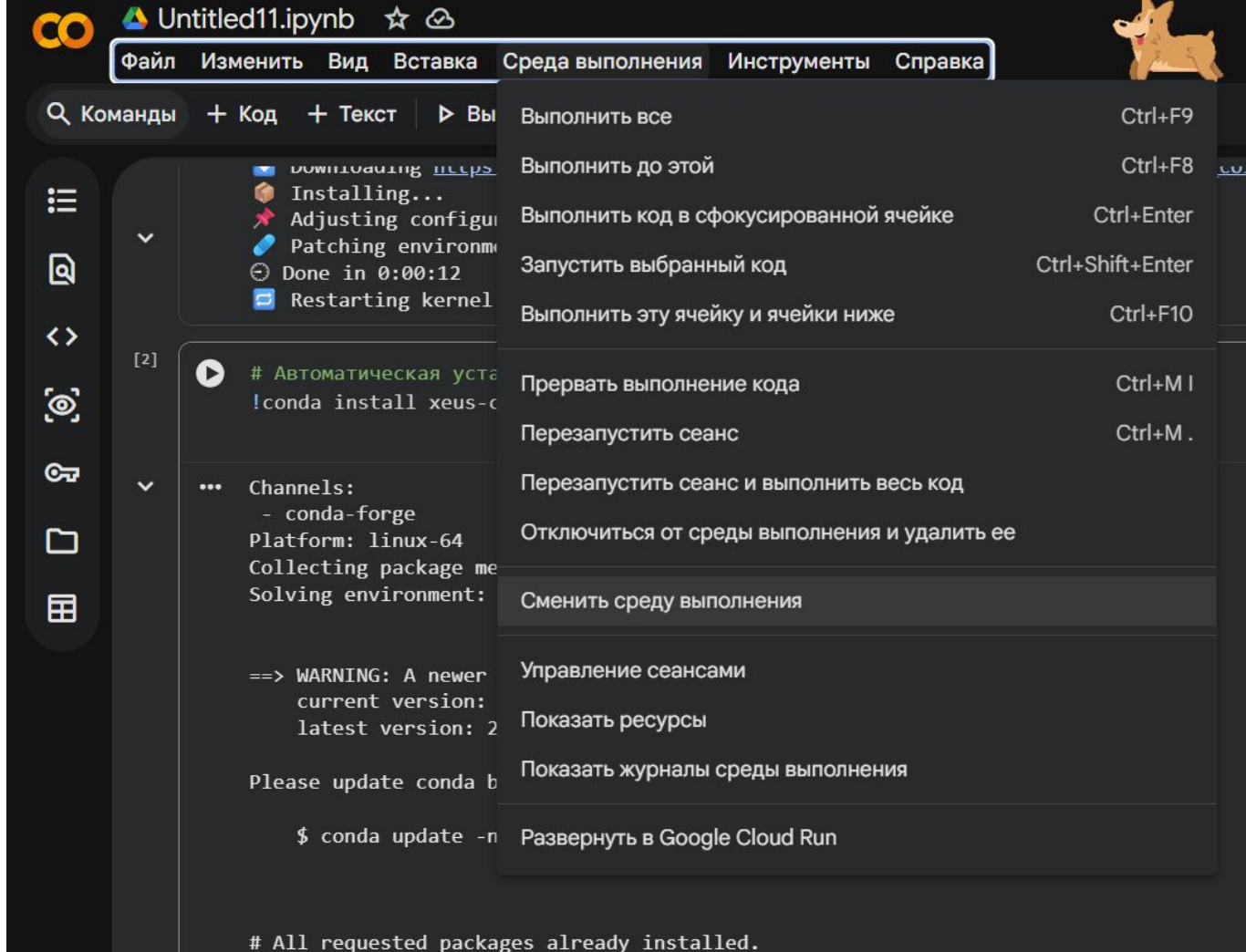
  $ conda update -n base -c conda-forge conda

# All requested packages already installed.
```

Завершение развертывания ядра xeus-cling: система подтверждает установку всех необходимых пакетов

Смена среды выполнения

Активация
установленного
интерпретатора через
меню "Среда
выполнения" ->
"Сменить среду
выполнения"



Смена среды выполнения

Активация
установленного
интерпретатора через
меню "Среда
выполнения" ->
"Сменить среду
выполнения"

Сменить среду выполнения

Тип среды выполнения

C++17

Python 3

R

Julia

C++14

C++11

C++17

еский процессор T4

сop H100

сop G4

сop A100

☐ Графический процессор L4

☐ TPU v5e-1

☐ TPU v6e-1

?

Нужен доступ к мощным графическим процессорам?
[Купите дополнительные вычислительные единицы](#)

Версия среды выполнения

?

Последняя версия (рекомендуется)

Отмена

Сохранить

Конфликт интерпретаторов: Ошибка `SyntaxError` при попытке ядра Python исполнить код C++

```
[3] 0 сек. #pragma cling add_include_path(".")

[4] 0 сек. #include <iostream>
          int a = 15;
          int b = 25;
          std::cout << "C++ Kernel is ALIVE! Sum: " << a + b << std::endl;

... File "/tmp/ipykernel_1555/3751990065.py", line 2
      int a = 15;
      ^
SyntaxError: invalid syntax
```

Переменные Терминал

Даже при смене Kernel в интерфейсе Colab, сессия может не переключить исполняемую среду (Runtime). Это требует использования "магических команд" или переинициализации путей Cling

Пути решения

```
with open('hello.cpp', 'w') as f:
    f.write('''
#include <iostream>

int main() {
    int a = 15;
    int b = 25;
    std::cout << "Sum: " << a + b << std::endl;

    return 0;
}
''')

# Компиляция и запуск
!g++ hello.cpp -o hello && ./hello
```

... Sum: 40

Использовать уже установленного `xeus-cling` напрямую через системный вызов

Это создаст файл, скомпилирует его интерпретатором и выведет результат